

0618 下午 Axios、跨域与 Vite 代理学习梳理

1. 这节课到底在学什么

0618 下午课件的主题是：Vue 项目如何和后端接口通信。

你可以把它理解成前端项目从“只会显示静态页面”，升级到“能向服务器要数据、处理错误、解决跨域”的阶段。

这节课有三条主线：

1. 用 [Axios](#) 发网络请求，拿到服务器返回的数据。
2. 用 [Axios 拦截器](#) 统一处理请求头、响应数据和错误。
3. 理解 [CORS 跨域](#) 的原因，并用 [Vite proxy](#) 在开发阶段解决跨域。

课后练习主要考察：

1. 能不能给响应拦截器补充 404、500、超时等错误处理。
2. 能不能在 [vite.config.js](#) 中配置多条代理规则。

2. 为什么要学 Axios

前端页面经常需要向服务器要数据，比如：

1. 获取用户信息。
2. 获取商品列表。
3. 提交登录表单。
4. 获取一句“打工人语录”。

传统写法可以用 [XMLHttpRequest](#) 或 [fetch](#)，但它们在企业项目里不够省心。

[fetch](#) 的常见麻烦：

1. 返回结果要手动 [response.json\(\)](#)。
2. 请求超时不够好统一处理。
3. 每个接口都要重复写错误处理。
4. 没有内置拦截器，统一加 Token 不方便。

[Axios](#) 的优势：

1. 自动把 JSON 转成 JS 对象。
2. 支持 [GET](#)、[POST](#) 等常见请求。
3. 支持请求超时。

- 4. 支持请求拦截器和响应拦截器。
- 5. 更适合项目里统一封装。

新手理解：

Axios 就像一个更好用的“网络请求工具箱”。你不用每次都从最底层开始拼请求，而是调用它封装好的方法。

3. GET 和 POST 怎么理解

GET

GET 常用于“向服务器拿数据”。

比如：

```
axios.get('https://apis.tianapi.com/dgryl/index', {  
  params: {  
    key: '你的接口密钥',  
  },  
})
```

这段代码最终会变成类似这样的地址：

```
https://apis.tianapi.com/dgryl/index?key=你的接口密钥
```

也就是说，`params` 里的参数会被拼到 URL 后面。

POST

POST 常用于“向服务器提交数据”。

比如：

```
axios.post('/login', {  
  username: 'xiaoming',  
  password: '123456',  
})
```

这里的数据通常不会直接显示在 URL 后面，而是放在请求体里。

简单记忆：

请求方式	常见用途	参数一般放哪里
GET	查询、获取数据	URL 后面的 query 参数
POST	新增、提交数据	请求体 body

4. Axios 基础请求结构

通用 GET 模板：

```
import axios from 'axios'

axios
  .get('目标接口地址', {
    params: {
      参数名: '参数值',
    },
  })
  .then((response) => {
    console.log(response.data)
  })
  .catch((error) => {
    console.error(error)
  })
```

重点理解：

1. `.then()` 表示请求成功后执行。
2. `.catch()` 表示请求失败后执行。
3. `response.data` 通常才是服务器返回的真正业务数据。

新手容易卡住的点：

`response` 不是最终数据，它是 Axios 包装过的一整个响应对象。里面有状态码、请求头、配置等信息。真正要展示到页面的数据，通常在 `response.data` 里。

5. 实战：调用“打工人语录”接口

课件使用的接口：

```
https://apis.tianapi.com/dgryl/index
```

调用前需要安装 Axios：

```
npm install axios
```

如果 PowerShell 不允许运行 `npm.ps1`，用：

```
npm.cmd install axios
```

示例代码核心逻辑：

```
<script setup>
import { ref } from 'vue'
import axios from 'axios'

const quoteText = ref('点击下方按钮，获取今日伴学打工人语录。')

function getWorkerQuote() {
```

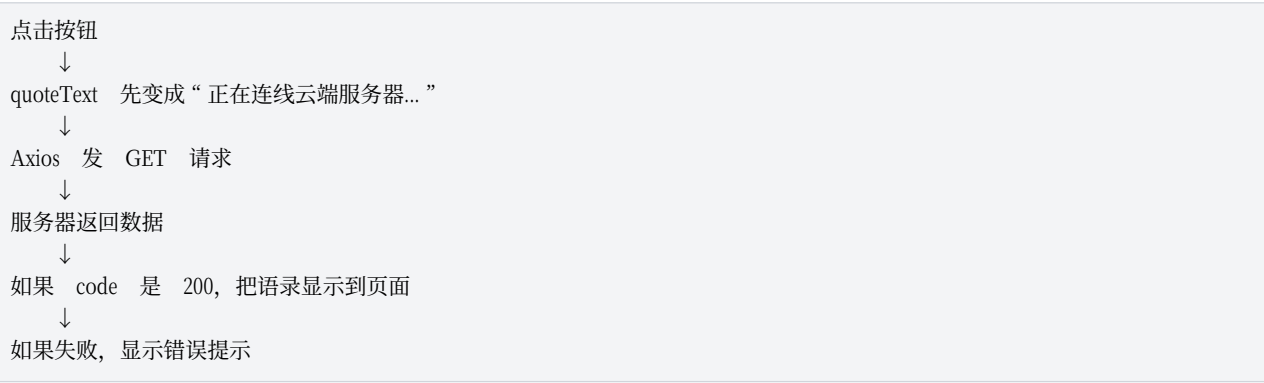
```
quoteText.value = '正在连线云端服务器...'

axios
  .get('https://apis.tianapi.com/dgryl/index', {
    params: {
      key: '你的接口密钥',
    },
  })
  .then((response) => {
    const result = response.data

    if (result.code === 200 && result.result) {
      quoteText.value = result.result.content
    } else {
      quoteText.value = `服务器异常：${result.msg}`
    }
  })
  .catch(() => {
    quoteText.value = '网络连接失败，请检查网络。'
  })
}
```

</script>

这段代码的运行顺序：



6. 为什么要封装 Axios

如果项目只有一个接口，直接在页面里写 `axios.get()` 没什么问题。

但真实项目里可能有几十个接口：

1. 每个接口都要写基础地址。
2. 每个接口都要写超时时间。
3. 每个接口都要加登录 Token。
4. 每个接口都要判断状态码。
5. 每个接口都要处理错误。

这会导致大量重复代码。

所以要把 Axios 封装成一个统一的请求工具，比如：

```
src/utils/request.js
```

以后页面里不再直接用原始 `axios`，而是用自己封装好的 `request`。

7. 拦截器到底是什么

课件里讲了两个拦截器：

1. 请求拦截器：请求发出去之前执行。
2. 响应拦截器：响应交给页面之前执行。

流程可以这样理解：

```
页面代码
↓
请求拦截器：统一加 Token、统一改请求配置
↓
服务器
↓
响应拦截器：统一拆 response.data、统一处理错误
↓
页面拿到最终数据
```

更直白一点：

请求拦截器管“出门前检查”，响应拦截器管“回来后拆包和验货”。

8. request.js 推荐模板

创建文件：

```
src/utils/request.js
```

可用于学习和练习的完整模板：

```
import axios from 'axios'

const service = axios.create({
  baseURL: '/api',
  timeout: 5000,
})

service.interceptors.request.use(
  (config) => {
    console.info('请求拦截器触发：准备发送请求')

    const myToken = 'fakeToken123456'
    config.headers.Authorization = `Bearer ${myToken}`

    return config
  },
  (error) => {
    return Promise.reject(error)
  },
)
```

```

service.interceptors.response.use(
  (response) => {
    console.info('响应拦截器触发：准备处理响应数据')

    const res = response.data

    if (res.code === 200) {
      return res
    }

    console.error(`业务执行失败：${res.msg}`)
    return Promise.reject(new Error(res.msg || '业务请求失败'))
  },
  (error) => {
    if (error.response) {
      const status = error.response.status

      if (status === 404) {
        console.error('请求的接口地址不存在')
      } else if (status === 500) {
        console.error('服务器内部崩溃故障')
      } else {
        console.error(`HTTP 请求异常，状态码：${status}`)
      }
    } else if (error.code === 'ECONNABORTED') {
      console.error('网络请求连接超时')
    } else {
      console.error('网络连接失败或请求被浏览器拦截')
    }

    return Promise.reject(error)
  },
)

export default service

```

新手注意：

`return config` 不能省略。请求拦截器里如果不返回 `config`，请求就无法继续发送。

`return Promise.reject(error)` 的意思是：把错误继续抛出去，让页面里的 `.catch()` 还能接住。

9. 页面中如何使用封装后的 request

```

<script setup>
import { ref } from 'vue'
import request from './utils/request.js'

const quoteText = ref('点击按钮测试封装后的 Axios 请求')

function getWorkerQuote() {
  const apiKey = '你的接口密钥'

  request
    .get(`/dgryl/index?key=${apiKey}`)
    .then((res) => {

```

```
        quoteText.value = res.result.content
      })
      .catch((err) => {
        quoteText.value = `请求异常：${err.message}`
      })
    }
  </script>
```

为什么这里不用 `res.data.result.content` ？

因为响应拦截器里已经做了：

```
return res
```

也就是已经把 `response.data` 这一层拆掉了。页面拿到的 `res`，直接就是后端返回的业务数据。

10. 什么是同源政策

同源政策是浏览器的安全规则。

判断两个地址是否同源，要同时看三件事：

- 1. 协议是否相同，比如 `http` 和 `https`。
- 2. 域名是否相同，比如 `localhost` 和 `127.0.0.1`。
- 3. 端口是否相同，比如 `5173` 和 `8080`。

三者全部相同，才叫同源。

示例：

当前页面地址	请求地址	是否同源	原因
<code>http://localhost:5173</code>	<code>http://localhost:5173/api/user</code>	是	协议、域名、端口都一致
<code>http://localhost:5173</code>	<code>https://localhost:5173/api/user</code>	否	协议不同
<code>http://localhost:5173</code>	<code>http://localhost:8080/api/user</code>	否	端口不同
<code>http://localhost:5173</code>	<code>http://127.0.0.1:5173/api/user</code>	否	域名字符串不同

11. CORS 跨域最重要的一句话

跨域不是请求发不出去。

更准确地说：

```
请求通常已经发到了服务器
服务器也可能已经返回了数据
但是浏览器检查响应头后，发现服务器没有允许当前页面读取这份数据
于是浏览器把响应拦下来，不交给你的 JS 代码
```

所以看到 CORS 报错时，不要第一反应以为“接口没请求到”。很多时候接口请求到了，只是浏览器不让前端代码读取结果。

浏览器主要看响应头里有没有类似：

```
Access-Control-Allow-Origin: http://localhost:5173
```

如果没有，浏览器就会报跨域错误。

12. Vite 代理为什么能解决开发期跨域

跨域限制发生在浏览器里。

服务器和服务器之间通信没有浏览器同源政策限制。

所以开发时可以让 Vite 开发服务器当中间人：

```
浏览器页面
  ↓ 请求 /api/dgryl/index
Vite 本地开发服务器
  ↓ 转发到 https://apis.tianapi.com/dgryl/index
远程服务器
  ↓ 返回数据给 Vite
Vite 再把数据返回给浏览器页面
```

对浏览器来说，它访问的是：

```
http://localhost:5173/api/dgryl/index
```

这是同源地址，所以浏览器不会按跨域拦截。

13. Vite 单条代理配置

打开：

```
vite.config.js
```

基础代理写法：

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'

export default defineConfig({
  plugins: [vue()],
  server: {
    proxy: {
      '/api': {
        target: 'https://apis.tianapi.com',
        changeOrigin: true,
        rewrite: (path) => path.replace(/^\/api/, ''),
      },
    },
  },
})
```

```
})
```

逐行理解：

配置	作用
<code>&#x27;/api&#x27;</code>	匹配所有以 <code>/api</code> 开头的请求
<code>target</code>	真正要转发到的后端服务器
<code>changeOrigin: true</code>	转发时修改请求来源，减少目标服务器拒绝的概率
<code>rewrite</code>	把本地路径里的 <code>/api</code> 去掉

路径变化示例：

```
前端请求：
/api/dgryl/index?key=xxx

Vite 转发成：
https://apis.tianapi.com/dgryl/index?key=xxx
```

14. 课后练习 1：封装错误处理拦截器

练习要求：

1. 打开 `src/utils/request.js`。
2. 找到响应拦截器的错误分支。
3. 判断 `404`、`500`、超时。
4. 故意把接口路径写错，观察控制台是否能输出对应错误。

参考实现核心代码：

```
(error) => {
  if (error.response) {
    const status = error.response.status

    if (status === 404) {
      console.error('请求的接口地址不存在')
    } else if (status === 500) {
      console.error('服务器内部崩溃故障')
    } else {
      console.error(`HTTP 请求异常，状态码：${status}`)
    }
  } else if (error.code === 'ECONNABORTED') {
    console.error('网络请求连接超时')
  } else {
    console.error('网络连接失败或请求被浏览器拦截')
  }

  return Promise.reject(error)
}
```

怎么测试 `404`：

把请求路径临时写错，例如：

```
request.get('/dgryl-wrong/index?key=${apiKey}')
```

点击按钮后打开浏览器控制台，看是否打印：

```
请求的接口地址不存在
```

15. 课后练习 2：配置多条代理规则

练习要求：

1. 打开 `vite.config.js`。
2. 配置 `/api-user` 转发到 `http://user.server.com`。
3. 配置 `/api-skill` 转发到 `http://skill.server.com`。
4. 两条规则都要做路径重写。
5. 写中文注释解释匹配逻辑。

可直接运行的参考代码：

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'

export default defineConfig({
  plugins: [vue()],
  server: {
    proxy: {
      // 匹配所有以 /api-user 开头的用户相关请求
      '/api-user': {
        // 将用户相关请求转发到用户测试服务器
        target: 'http://user.server.com',
        // 修改请求来源，让目标服务器认为请求来自代理服务器
        changeOrigin: true,
        // 转发前移除 /api-user 前缀，避免后端收到多余路径
        rewrite: (path) => path.replace(/^\/api-user/, ''),
      },
      // 匹配所有以 /api-skill 开头的技能相关请求
      '/api-skill': {
        // 将技能相关请求转发到技能测试服务器
        target: 'http://skill.server.com',
        // 修改请求来源，减少开发期跨域和来源校验问题
        changeOrigin: true,
        // 转发前移除 /api-skill 前缀，得到后端真实接口路径
        rewrite: (path) => path.replace(/^\/api-skill/, ''),
      },
    },
  },
})
```

路径变化示例：

前端请求：
/api-user/profile

实际转发：
http://user.server.com/profile

前端请求：
/api-skill/list

实际转发：
http://skill.server.com/list

16. 新手最容易踩的坑

问题	原因	解决
axios 引入报错	没安装依赖	执行 <code>npm.cmd install axios</code>
<code>npm run dev</code> 找不到 <code>package.json</code>	终端没进入项目根目录	先 <code>cd D:\code\2026.06-07\vue3-demo-project\vue3-demo-project</code>
页面一直显示加载中	<code>.catch()</code> 没有更新页面状态	在 <code>.catch()</code> 里给用户提示
CORS 报错	浏览器禁止读取跨域响应	开发期配置 Vite 代理，生产环境让后端配置 CORS
代理配置后仍没效果	没重启 Vite 服务	修改 <code>vite.config.js</code> 后重启 <code>npm.cmd run dev</code>
请求地址还是远程域名	<code>baseUrl</code> 没改成 <code>/api</code>	检查 <code>request.js</code> 的 <code>baseUrl</code>
页面里拿不到 <code>res.result</code>	响应拦截器没有 <code>return res</code>	检查响应拦截器返回值
修改了代理但 Network 看不到 <code>/api</code>	页面请求没有走封装的 <code>request</code>	检查导入的是不是 <code>request.js</code>

17. 推荐学习顺序

1. 先能用原始 `axios.get()` 请求接口。
2. 再理解 `response.data` 是什么。
3. 再把 Axios 封装到 `src/utlis/request.js`。
4. 再理解请求拦截器和响应拦截器分别发生在什么时候。
5. 再学同源政策：协议、域名、端口必须全部相同。
6. 再学 CORS：请求不是没发出，而是响应被浏览器拦了。
7. 最后学 Vite proxy：让浏览器请求同源的 `/api`，由 Vite 转发到远程服务器。

18. 操作检查表

完成这节课后，你应该能做到：

1. 在项目中安装 `axios`。

2. 在 `App.vue` 里点击按钮请求接口。
3. 看懂 `.then()` 和 `.catch()` 的作用。
4. 创建 `src/utils/request.js` 并封装 Axios。
5. 在请求拦截器里统一加请求头。
6. 在响应拦截器里统一拆数据和处理错误。
7. 说清楚什么是同源。
8. 说清楚 CORS 为什么不是“请求没发出去”。
9. 在 `vite.config.js` 里配置代理。
10. 修改代理配置后知道要重启开发服务器。

如果你能把“浏览器 -> Vite 代理 -> 远程服务器 -> Vite 代理 -> 浏览器”这个链路讲清楚，这节课最难的跨域和代理部分就已经理解到位了。